

Universidade do Algarve
Faculdade de Ciências e Tecnologia

Laboratório de Engenharia de Software
Ano letivo 2025/2026

Spot-On

Sistema de Gestão de Espaços de Estudo

Relatório Técnico do Trabalho Prático

Docentes: José Barateiro
Marco Giunti

Equipa: André Martins – n.º a35463
António Matoso – n.º a84100
Tomás Machado – n.º a84014
Vasco Hilário – n.º 83993

23 de maio de 2026

Conteúdo

1	Introdução	2
2	Visão Geral do Sistema	2
2.1	Funcionalidades Principais	2
3	Atores do Sistema	3
4	Requisitos	3
4.1	Requisitos Funcionais	3
4.2	Requisitos de Qualidade	4
5	Arquitetura e Stack Tecnológica	4
5.1	Visão de Alto Nível	4
5.2	Camadas Lógicas	4
5.3	Sumário da Stack e Alternativas	5
5.4	Decisões Técnicas e Justificações	7
5.4.1	1. Core Framework: Next.js & TypeScript	7
5.4.2	2. Base de Dados e ORM: PostgreSQL & Prisma	7
5.4.3	3. Estratégia de Autenticação e Gestão de Identidade	7
5.4.4	4. Modelação de Dados e Resolução de Conflitos	8
5.4.5	5. Infraestrutura: Docker	9
6	Modelo de Dados	9
6.1	Entidades de Domínio	10
6.2	Diagrama do Modelo de Dados	10
6.3	Decisões de Modelação Relevantes	10
6.4	Excerto do <i>Schema</i> Prisma	11
7	Backend e API	11
8	Controlo de Versões	12
9	Gestão de Projeto – SCRUM	12
10	Testes e Integração Contínua	12
10.1	Testes Automatizados	12
10.2	Pipeline de CI	13
11	Deployment	13
11.1	Execução Local	13
11.2	Ambiente de Produção	13
12	Conclusão	14

1 Introdução

O presente relatório documenta o trabalho prático desenvolvido no âmbito da unidade curricular de Laboratório de Engenharia de Software, no ano letivo 2025/2026. O projeto, denominado **Spot-On**, consiste num sistema de gestão de espaços de estudo, concebido para o contexto da biblioteca da Universidade do Algarve.

A motivação para o tema surge de uma necessidade frequente entre estudantes: identificar rapidamente que mesas individuais ou salas de grupo se encontram disponíveis num determinado momento, sem percorrer fisicamente todos os pisos da biblioteca. O Spot-On disponibiliza essa informação em tempo real através de uma interface visual baseada nas plantas dos pisos, permitindo ainda organizar sessões de estudo, convidar colegas, e gerir a ocupação dos espaços com recurso a códigos QR.

Este documento descreve a visão geral do sistema, os atores envolvidos, os requisitos funcionais e de qualidade, a arquitetura e o modelo de dados adotados, bem como a justificação das principais decisões técnicas.

2 Visão Geral do Sistema

O Spot-On é uma aplicação web que permite a alunos da Universidade do Algarve visualizar e gerir o uso de espaços de estudo na biblioteca. O sistema disponibiliza uma representação visual dos pisos da biblioteca, sobre a qual são marcados os espaços (mesas individuais e salas de grupo), com indicação do seu estado de ocupação em tempo real.

2.1 Funcionalidades Principais

- **Autenticação institucional** – acesso restrito a contas de correio @ualg.pt, com autenticação *passwordless* via *magic link* enviado por email;
- **Visualização de plantas** – exibição interativa das plantas dos pisos da biblioteca, com sobreposição dos espaços de estudo;
- **Gestão de espaços** – mesas individuais e salas de grupo, com atributos como capacidade, presença de tomada elétrica, computador ou quadro interativo;
- **Sessões de estudo** – criação, consulta e encerramento de sessões, associadas a um espaço e a um anfitrião (*host*);
- **Confirmação de presença via QR code** – cada espaço dispõe de um *token* QR rotativo que valida fisicamente a ocupação;
- **Convites e participações** – o anfitrião pode convidar outros utilizadores para a sessão; estes confirmam ou rejeitam o convite;
- **Reports** – mecanismo para reportar comportamentos incorretos, com confirmação por outros utilizadores e janela temporal limitada;
- **Notificações** – sistema in-app para pedidos de adesão, pedidos de prova de presença e outros eventos relevantes;
- **Gamificação** – atribuição automática de pontos e *badges* (mensais, anuais ou permanentes) que premeiam a utilização regular e correta do sistema;
- **Administração** – gestão de plantas, espaços, *badges* e moderação de utilizadores (incluindo banimento temporário).

3 Atores do Sistema

Foram identificados dois atores humanos no sistema:

Ator	Descrição
Aluno	Utilizador final do sistema. Pode autenticar-se, consultar a disponibilidade de espaços, criar e gerir sessões de estudo (anfitrião), participar em sessões de outros (convidado), submeter <i>reports</i> , receber notificações e acumular pontos e <i>badges</i> .
Administrador	Responsável pela gestão da configuração do sistema. Define plantas dos pisos, cria e edita espaços, configura <i>badges</i> e modera utilizadores.

Tabela 1: Atores do sistema Spot-On.

4 Requisitos

4.1 Requisitos Funcionais

A tabela seguinte sumariza os requisitos funcionais identificados, agrupados por área. A lista não é exaustiva, mas reflete o conjunto efetivamente implementado.

ID	Área	Descrição
RF-01	Autenticação	O sistema autentica utilizadores via <i>magic link</i> enviado para o seu email institucional.
RF-02	Autenticação	O sistema rejeita utilizadores cujo email não pertença ao domínio @ualg.pt.
RF-03	Espaços	O sistema apresenta as plantas dos pisos com a localização dos espaços.
RF-04	Espaços	O sistema indica em tempo real o estado de ocupação de cada espaço.
RF-05	Espaços	O utilizador pode filtrar espaços por características (capacidade, tomada, computador, quadro).
RF-06	Sessões	Um utilizador pode iniciar uma sessão de estudo num espaço disponível.
RF-07	Sessões	Um anfitrião pode convidar outros utilizadores para a sessão.
RF-08	Sessões	Um utilizador pode aceitar ou rejeitar um convite recebido.
RF-09	Sessões	A presença é confirmada por leitura do QR code do espaço.
RF-10	Reports	Um utilizador pode submeter um <i>report</i> sobre uma sessão.
RF-11	Reports	Outros utilizadores podem confirmar um <i>report</i> , num prazo limitado.
RF-12	Notificações	O sistema envia notificações in-app para eventos relevantes.
RF-13	Gamificação	O sistema atribui pontos e <i>badges</i> automaticamente.
RF-14	Admin	O administrador gere plantas, espaços, <i>badges</i> e utilizadores.
RF-15	Admin	O administrador pode banir temporariamente um utilizador.

Tabela 2: Requisitos funcionais do Spot-On.

4.2 Requisitos de Qualidade

Os requisitos de qualidade (não funcionais) descrevem propriedades transversais ao sistema, não associadas a funcionalidades específicas. A tabela seguinte enumera os requisitos identificados, agrupados por categoria ISO/IEC 25010, com critérios verificáveis sempre que possível.

ID	Categoria	Descrição e Critério
RQ-01	Segurança	O acesso ao sistema é restrito a endereços de email do domínio @ualg.pt; tentativas com outros domínios são rejeitadas no <i>backend</i> .
RQ-02	Segurança	Os <i>tokens</i> de <i>magic link</i> expiram em 5 minutos e só podem ser utilizados uma vez.
RQ-03	Segurança	O <i>token</i> QR de cada espaço é rotacionado periodicamente, impedindo a reutilização de fotografias antigas para validar presença.
RQ-04	Segurança	Utilizadores banidos (bannedUntil no futuro) não conseguem iniciar sessão nem participar em sessões.
RQ-05	Manutenibilidade	A camada de persistência é totalmente acedida através de um ORM, isolando o restante código de detalhes do SGBD.
RQ-06	Manutenibilidade	Todas as alterações ao modelo de dados são versionadas como migrações no repositório (prisma/migrations).
RQ-07	Portabilidade	A aplicação executa em qualquer máquina com Docker instalado, através de um único comando (docker compose up), sem dependências locais adicionais.
RQ-08	Fiabilidade	A <i>pipeline</i> de CI executa, em cada <i>push</i> e <i>pull request</i> , análise estática (<i>lint</i>), testes automatizados e <i>build</i> de produção; nenhum <i>merge</i> em main ocorre sem que estes passos passem.
RQ-09	Usabilidade	A consulta de disponibilidade é feita visualmente sobre a planta do piso, sem necessidade de o utilizador conhecer identificadores de espaços.
RQ-10	Compatibilidade	A aplicação é responsiva e funciona em navegadores modernos (<i>desktop</i> e <i>mobile</i>), incluindo os tablets fixados nos espaços para exibir o QR code.

Tabela 3: Requisitos de qualidade do Spot-On.

5 Arquitetura e Stack Tecnológica

A arquitetura do Spot-On foi desenhada com foco na **segurança, integridade dos dados e manutenibilidade**, utilizando uma *stack* moderna baseada em TypeScript. Esta secção descreve a visão de alto nível, as camadas lógicas, e as principais decisões técnicas com as respetivas justificações.

5.1 Visão de Alto Nível

O Spot-On segue uma arquitetura **cliente-servidor** com aplicação web *full-stack* construída sobre Next.js. O *frontend* (React) e o *backend* (API *routes* do Next.js) coabitam no mesmo projeto, com responsabilidades claramente separadas. A persistência é assegurada por uma base de dados PostgreSQL, acedida exclusivamente através do ORM Prisma. A autenticação é delegada à biblioteca NextAuth.js, que gere o fluxo de *magic link* sobre o domínio institucional @ualg.pt.

O diagrama de componentes detalhado da arquitetura está disponível no repositório em `diagrams/component/backend_architecture.drawio`.

5.2 Camadas Lógicas

Internamente, a aplicação Next.js encontra-se organizada em três camadas:

1. **Apresentação** – componentes React responsáveis pela interface, incluindo a renderização das plantas, formulários e *dashboards*. Faz uso de *Server Components* e *Client Components* consoante a necessidade de interatividade.
2. **API e regras de negócio** – *API routes* do Next.js, onde reside a validação de dados, as regras de domínio (e.g., elegibilidade para iniciar sessão, validade do QR *token*, atribuição de pontos) e a orquestração das operações. Um *middleware* global (`proxy.ts`) protege as rotas autenticadas, verificando a presença do *cookie* de sessão NextAuth e redirecionando para a página de *sign-in* quando ausente.
3. **Persistência** – camada de acesso a dados totalmente delegada ao Prisma, que gera um cliente tipado a partir do *schema* `schema.prisma`.

5.3 Sumário da Stack e Alternativas

A tabela seguinte resume as tecnologias adotadas e as principais alternativas ponderadas.

Camada	Tecnologia	Alternativa	Justificação
<i>Framework full-stack</i>	Next.js (App Router, React)	React + Express separados	Unifica UI e API num só projeto, simplifica <i>deployment</i> e tipagem partilhada.
Linguagem (<i>frontend</i> e <i>backend</i>)	TypeScript	JavaScript	Mesma linguagem em ambas as vertentes, com tipagem estática que reduz erros e integra-se com o cliente Prisma gerado.
ORM	Prisma	TypeORM, Sequelize	<i>Schema</i> declarativo, migrações automáticas, tipagem forte ponta-a-ponta.
Base de dados	PostgreSQL	MySQL, MongoDB	Modelo relacional adequado às entidades; enum types nativos; maturidade.
Autenticação	NextAuth.js (<i>magic link</i>)	Auth0, Clerk, implementação própria	Integração nativa com Next.js, fluxo <i>passwordless</i> pronto a usar.
Envio de email	Nodemailer / Resend	SMTP direto, AWS SES	API simples para envio transacional dos <i>magic links</i> .
Containerização	Docker + Docker Compose	VM, <i>deploy</i> direto	<i>Setup</i> reproduzível em qualquer máquina com um único comando.
Testes	Jest + Postman/Newman	Vitest, Mocha, Playwright	Jest para unidade/integração; Newman para testes <i>end-to-end</i> da API.
CI	GitHub Actions	GitLab CI, CircleCI	Integração nativa com o repositório, <i>free tier</i> suficiente.
Gestão	Jira	Trello, ClickUp	Suporte completo a SCRUM (<i>backlog</i> , <i>sprints</i> , <i>boards</i>).

Tabela 4: Stack tecnológica e alternativas consideradas.

5.4 Decisões Técnicas e Justificações

Apresentam-se de seguida as cinco decisões técnicas estruturantes do projeto, com a respetiva justificação.

5.4.1 1. Core Framework: Next.js & TypeScript

Optou-se pelo **Next.js (App Router)** como *framework* principal, suportado por TypeScript.

- **Justificação:** Esta escolha permite uma arquitetura *full-stack* num único repositório, onde o *frontend* (React) e o *backend* (API Routes) partilham os mesmos tipos de dados. O uso de TypeScript foi crucial para garantir a robustez do código, prevenindo erros de tipagem comuns durante o desenvolvimento das funcionalidades de reservas e autenticação. Adicionalmente, a experiência prévia da equipa com estas tecnologias foi um fator decisivo, permitindo mitigar riscos técnicos e acelerar a implementação do sistema.
- **Alternativas ponderadas:** manter *frontend* (React) e *backend* (Express) em projetos independentes. Foi descartada por implicar gerir *cross-origin requests* entre dois servidores, duplicar configuração de *deployment*, e introduzir etapas adicionais de geração de tipos entre cliente e servidor.

5.4.2 2. Base de Dados e ORM: PostgreSQL & Prisma

Para a persistência de dados, optámos por **PostgreSQL** orquestrado pelo **Prisma ORM**.

- **PostgreSQL:** escolhido pela sua fiabilidade e suporte robusto a relações complexas (*ACID compliance*), essenciais para um sistema de reservas onde a consistência é crítica. O modelo do Spot-On é fortemente relacional e faz uso intensivo de tipos enumerados (`SessionStatus`, `JoinRequestStatus`, `ReportStatus`, `NotificationType`, etc.), nativamente suportados em PostgreSQL.
- **Prisma ORM:** justifica-se pela *type safety* automática. O ficheiro `schema.prisma` serve como fonte única de verdade, quer para a geração do cliente tipado, quer para a geração de migrações SQL (`prisma migrate dev`). Em comparação com abordagens baseadas em decoradores (TypeORM) ou SQL puro, oferece menor curva de aprendizagem e ciclos de evolução do modelo mais rápidos e menos propensos a erros.
- **Exemplo prático:** as migrações automáticas permitiram-nos evoluir o esquema (adicionando, por exemplo, o campo `studentId` ao `User` ou o modelo `JoinRequest`) mantendo o histórico de versões sincronizado entre toda a equipa, sob o diretório `prisma/migrations`.

5.4.3 3. Estratégia de Autenticação e Gestão de Identidade

Para a gestão de identidade, implementou-se o **NextAuth.js** (com o `@next-auth/prisma-adapter`). A arquitetura de autenticação sofreu uma iteração importante durante o desenvolvimento, adaptando-se às restrições da infraestrutura da universidade.

Decisão Arquitetural: do SSO (Azure AD) para *Passwordless (Magic Links)*.

- **Tentativa inicial (SSO via Microsoft Entra ID):** o planeamento inicial previa a implementação de um verdadeiro *Single Sign-On* (SSO) utilizando o *provider* do Azure AD no NextAuth. Como a comunidade académica utiliza o ecossistema Microsoft (Outlook/Office 365), o objetivo era proporcionar um *login* direto e fluido (“Continuar com a conta da Universidade”).

- **Bloqueio técnico (restrições de *tenant*):** durante a implementação, a equipa deparou-se com um erro 401 (*Unauthorized*) no portal do Azure (Figura 1). Concluiu-se que as políticas de segurança dos Serviços de Informática da UAlg bloqueiam ativamente o registo de novas aplicações (*App Registrations*) por parte dos alunos no *tenant* principal da universidade.
- **Solução de contingência (*Magic Links*):** face à impossibilidade de integração OAuth/-SAML direta sem intervenção demorada da administração de TI da UAlg, a equipa adotou uma estratégia de autenticação *passwordless* (*magic links* via *Email Provider* do NextAuth, com envio através de Nodemailer/Resend).

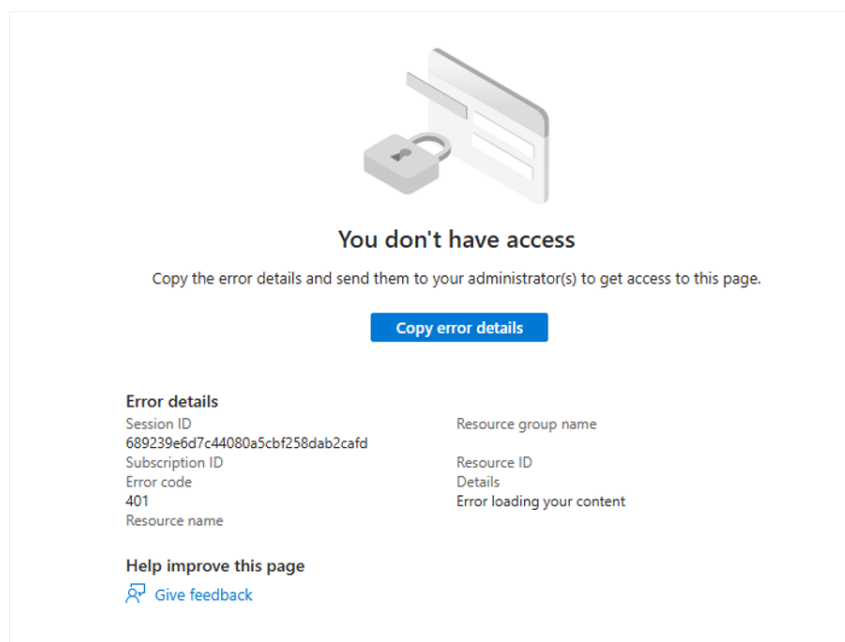


Figura 1: Erro 401 (*Unauthorized*) no portal do Azure ao tentar registar a aplicação no *tenant* da UAlg.

Justificação da solução implementada.

- **Independência de infraestrutura:** o sistema funciona de forma autónoma, sem depender de autorizações de terceiros (Serviços de Informática) para operar.
- **Exclusividade e validação de domínio:** o *login* é intercetado e o domínio do email validado por *regex*, garantindo que apenas endereços terminados em *@ualg.pt* recebem o link de acesso e conseguem criar conta (controlado pela variável de ambiente *ALLOWED_EMAIL_DOMAIN*).
- **Segurança:** mantém-se o princípio fundamental de não armazenar *hashes* de *passwords* na base de dados (a tabela *User* não possui campo *password*), mitigando o risco de fugas de dados. Os *tokens* de *magic link* expiram em 5 minutos e só podem ser utilizados uma vez.
- **Experiência de utilizador (UX):** o fluxo de *magic link* atua como um SSO simplificado — o utilizador não precisa de memorizar uma nova *password*, bastando clicar no link seguro enviado para a caixa de correio institucional.

5.4.4 4. Modelação de Dados e Resolução de Conflitos

Durante a modelação, surgiram desafios de nomenclatura e de *design* que foram resolvidos através de uma separação clara de conceitos no *schema.prisma*:

- **Auth vs. negócio:**
 - **Session** (*Auth*) – mantida estritamente para a gestão de sessões de *login* (*cookies*) do NextAuth.
 - **StudySession** (negócio) – criada para representar as sessões de estudo ou reservas de espaços. Esta distinção evita conflitos no código e clarifica o propósito de cada tabela.
- **Identificação do estudante:** adicionou-se o campo `studentId` ao modelo de utilizador para permitir a futura integração com cartões de estudante físicos ou sistemas académicos, separando o ID interno do sistema (CUID) do número de aluno.
- **Participação vs. pedido de adesão:** para modelar corretamente o ciclo de vida de um convite, foram introduzidas duas entidades distintas:
 - **JoinRequest** – representa um pedido de adesão a uma sessão, com estado (`PENDING`, `ACCEPTED`, `REJECTED`).
 - **UserOnStudySession** – tabela-pivô com chave composta (`userId`, `sessionId`) que regista a participação *efetiva* de um utilizador numa sessão, apenas depois de o pedido ser aceite.
- **Relações com flexibilidade futura:** estabeleceu-se uma relação 1:*N* entre **User** e **Account**, permitindo que um mesmo utilizador venha a associar múltiplos *providers* de autenticação (e.g., se o SSO institucional se tornar viável no futuro).

5.4.5 5. Infraestrutura: Docker

O ambiente de desenvolvimento é padronizado via **Docker** (observável pelos ficheiros `Dockerfile` e `compose.yaml`).

- **Justificação:** garante que todos os membros da equipa executam a mesma versão da base de dados PostgreSQL, eliminando erros do tipo “funciona na minha máquina”. O arranque completo do ambiente faz-se com um único comando (`docker compose up`).

Resumo. Esta arquitetura privilegia a **segurança** (ao remover *passwords* e validar o domínio institucional), a **integridade** (via PostgreSQL + Prisma com migrações versionadas) e a **manutenibilidade** (TypeScript + Next.js com tipagem partilhada), criando uma base sólida para o desenvolvimento das funcionalidades de gestão de espaços da UAlg.

6 Modelo de Dados

A persistência é definida no *schema* Prisma (`spot-on/prisma/schema.prisma`). As entidades dividem-se em dois grupos: as relativas à autenticação (**User**, **Account**, **Session**, **VerificationToken**, herdadas do modelo do NextAuth.js) e as do domínio Spot-On.

6.1 Entidades de Domínio

Entidade	Descrição
User	Utilizador autenticado. Inclui <code>studentId</code> opcional, pontos acumulados (<code>points</code>) e prazo de banimento (<code>bannedUntil</code>), se aplicável.
FloorPlan	Planta de um piso da biblioteca; serve de <i>canvas</i> sobre o qual os espaços são posicionados.
Space	Espaço de estudo (mesa individual ou sala de grupo). Inclui geometria (<i>points</i> SVG), capacidade, atributos (tomada, computador, quadro interativo) e o QR rotativo.
StudySession	Sessão de estudo num espaço, com anfitrião, hora prevista de fim e estado (<code>ACTIVE</code> , <code>COMPLETED</code> , <code>EXPIRED</code>).
JoinRequest	Pedido de adesão de um utilizador a uma sessão de estudo, com estado (<code>PENDING</code> , <code>ACCEPTED</code> , <code>REJECTED</code>).
UserOnStudySession	Tabela-pivô que regista a participação efetiva de um utilizador numa sessão (após aceitação do pedido), com <i>timestamp</i> de adesão.
Report	<i>Report</i> submetido por um utilizador sobre uma sessão, com janela de confirmação de 10 minutos.
ReportConfirmation	Confirmação de um <i>report</i> por um terceiro utilizador.
Notification	Notificação <i>in-app</i> dirigida a um utilizador (e.g., pedido de adesão, prova de presença, sessão a expirar).
Badge	Insígnia que pode ser atribuída (mensal, anual ou permanente).
UserBadge	Tabela-pivô que regista a atribuição de <i>badges</i> a utilizadores.

Tabela 5: Entidades do modelo de domínio Spot-On.

O modelo recorre ainda a vários *enum types* nativos do PostgreSQL: `SessionStatus`, `JoinRequestStatus`, `SpaceType`, `SpaceShape`, `ReportStatus`, `NotificationType` e `NotificationStatus`.

6.2 Diagrama do Modelo de Dados

O diagrama UML de classes do domínio está disponível em `diagrams/class/domain_class_diagram.drawio`, mantido em formato *drawio* para facilitar a sua edição em conjunto com a evolução do *schema*.

6.3 Decisões de Modelação Relevantes

- **Geometria dos espaços armazenada como *string* SVG.** O atributo `points` de `Space` segue o formato "`x1,y1 x2,y2 ...`", compatível com o elemento `<polygon>` do SVG. Esta decisão evita a sobrecarga de uma tabela auxiliar de vértices e permite *rendering* direto na interface.
- **Token QR rotativo.** O atributo `currentQrToken` é único e atualizado periodicamente por uma rotina do servidor (*cron job* com `node-cron`), impedindo que um QR fotografado uma única vez possa ser reutilizado posteriormente.
- **Janela de confirmação de *reports*.** O campo `timeToConfirm` usa o valor por omissão gerado na base de dados (`now() + interval '10 minutes'`), garantindo coerência mesmo perante dessincronização horária no *frontend*.

- **Separação JoinRequest / UserOnStudySession.** O ciclo de vida de um convite é modelado em duas tabelas distintas: a primeira regista a intenção de participar (com estado evolutivo), a segunda regista apenas a participação confirmada. Esta separação evita ambiguidade sobre o estado atual e simplifica as consultas.
- **Tabelas-pivô explícitas com chave composta.** UserOnStudySession, ReportConfirmation e UserBadge são modeladas como entidades próprias por carregarem informação adicional (joinedAt, confirmedAt, awardedAt).
- **Banimento como DateTime.** User.bannedUntil guarda diretamente o instante até ao qual o banimento é válido, eliminando a necessidade de um *flag* associado a uma data calculada.

6.4 Excerto do *Schema* Prisma

A título ilustrativo, apresenta-se a definição da entidade central *Space*:

```
model Space {
  id          String      @id @default(cuid()) @db.VarChar(30)
  floorPlanId String      @db.VarChar(30)
  name        String      @db.VarChar(255)
  points      String      /// pontos SVG do polígono
  capacity    Int
  currentQrToken String    @unique @db.VarChar(255)
  description String?     @db.VarChar(255)
  hasPowerOutlet Boolean   @default(false)
  hasComputer Boolean      @default(false)
  hasInteractiveBoard Boolean @default(false)
  type        SpaceType   @default(INDIVIDUAL_DESK)
  shape       SpaceShape?

  createdAt DateTime @default(now())
  updatedAt DateTime @default(now()) @updatedAt

  floorPlan    FloorPlan    @relation(fields: [floorPlanId], references: [id],
    onDelete: Cascade)
  sessions     StudySession[]
  joinRequests JoinRequest[]
}
```

7 Backend e API

O *backend* é exposto através das *API routes* do Next.js, sob */api/**. A API segue um estilo **REST**, com *endpoints* organizados por recurso. Os grupos atualmente disponíveis são:

- */api/auth/** – *endpoints* geridos pelo NextAuth.js (envio de *magic links*, *callbacks*, *sign-out*).
- */api/spaces/** – gestão e consulta de espaços, plantas e sessões de estudo associadas (incluindo criação, adesão, prolongamento, *check-out* e *reports*).
- */api/qrcode/** – emissão e validação dos *tokens* QR rotativos dos espaços.
- */api/notifications/** – consulta e atualização de notificações *in-app*.
- */api/leaderboard* – exposição dos rankings de pontuação para a componente de gamificação.
- */api/user/** – consulta e atualização do perfil do utilizador autenticado.

Cada *endpoint* obtém a sessão via `getSession`, valida os dados de entrada e delega o acesso a dados ao Prisma *Client*. A proteção de rotas de UI é complementada por um *middleware* (`proxy.ts`) que valida a presença do *cookie* de sessão antes de servir páginas privadas.

A persistência é totalmente versionada por migrações Prisma. A cada alteração do *schema*, é gerada uma nova migração SQL com `npx prisma migrate dev`, ficando registada no diretório `prisma/migrations` (à data, 10 migrações cobrindo desde a criação inicial do *schema* até à introdução do modelo `JoinRequest` e do campo `bannedUntil`). Em produção, as migrações são aplicadas no *deploy* via `prisma migrate deploy`.

8 Controlo de Versões

Todo o desenvolvimento é gerido em Git, com repositório alojado no GitHub e partilhado em modo de leitura com os docentes. Cada elemento da equipa contribui sob a sua identidade pessoal, garantindo que o histórico reflete fielmente a contribuição individual.

Fluxo adotado. Foi seguido um modelo simplificado baseado em *feature branches*:

1. Cada *user story* ou tarefa origina um *branch* dedicado (`feature/<descrição>`);
2. A integração no *branch* principal (`main`) ocorre exclusivamente via *pull request*;
3. Cada *pull request* requer revisão por outro elemento da equipa antes de *merge*;
4. A *pipeline* de CI deve passar (*lint*, testes, *build*) como pré-condição.

9 Gestão de Projeto – SCRUM

A gestão do projeto seguiu a metodologia SCRUM, suportada pela plataforma Jira. Definiu-se um *product backlog* composto por *user stories* com critérios de aceitação claros. As *stories* foram refinadas e priorizadas no início de cada *sprint*.

Cerimónias. Cada *sprint* (com duração de **duas** semanas) inclui:

- ***Sprint planning*** – seleção das *stories* a incluir no *sprint*;
- ***Daily*** – ponto de situação curto entre os elementos;
- ***Sprint review*** – demonstração do incremento;
- ***Retrospectiva*** – análise do que correu bem, do que deve melhorar e ações a tomar.

Evidências. O Jira mantém o histórico de *sprints*, cartões movidos entre estados (*To Do*, *In Progress*, *In Review*, *Done*), e os *burndown charts* associados.

10 Testes e Integração Contínua

10.1 Testes Automatizados

A estratégia de testes combina **Jest** (executado quer localmente quer na *pipeline* de CI) com uma *suite* **Postman/Newman** adicional para validação *end-to-end* da API. O código de teste está organizado da seguinte forma:

- **Testes unitários** (`spot-on/__tests__/unit/`) – cobrem funções isoladas de lógica de negócio, e.g., utilitários de autenticação (`auth-utils.test.ts`), geração e validação dos *tokens* QR dinâmicos (`dynamic-qrcode.test.ts`), regras de expiração de *reports* (`ReportExpiry.test.ts`) e notificações.
- **Testes de integração** (`spot-on/__tests__/integration/`) – exercitam os *endpoints* da API contra uma instância real de PostgreSQL e um servidor Next.js em execução. Cobrem os fluxos críticos: sessões de estudo (criação, adesão, prolongamento, libertação, expiração), *reports* ativos, QR code, leaderboard, espaços e *badges*.
- **Testes *end-to-end* da API** (`spot-on/postman/`) – coleção Postman executável via Newman, com *seed* dedicado (`seed-test.ts`) que prepara o estado da base de dados antes da execução.

10.2 Pipeline de CI

A *pipeline* é definida em `.github/workflows/ci.yml` e dispara em cada *push* para *main*, *develop* ou *branches* `feature/**/fix/**`, bem como em cada *pull request* para *main* ou *develop*. Estão definidos três *jobs* encadeados:

1. **Lint** – análise estática com ESLint (`npm run lint`).
2. **Test** – arranca um serviço PostgreSQL 17, gera o cliente Prisma (`npx prisma generate`), levanta um servidor Next.js de teste em `localhost:3001` e executa a *suite* Jest unitária e de integração (`npm test`).
3. **Build** – depende de *lint* e *test*, instala dependências e executa `npm run build` (`next build`) para garantir que a aplicação compila para produção sem erros.

Adicionalmente, o workflow `codeql.yml` executa análise de segurança estática (*CodeQL*) sobre o código TypeScript. A passagem de todos os *jobs* é pré-condição obrigatória para a aprovação dos *pull requests*.

11 Deployment

11.1 Execução Local

A execução local recorre a **Docker Compose**, com dois serviços: a aplicação Next.js e a instância de PostgreSQL. Após clonar o repositório, a aplicação é colocada em execução com um único comando:

```
docker compose up --build
```

As migrações da base de dados são aplicadas automaticamente no arranque do *container*.

11.2 Ambiente de Produção

Para o ambiente de produção, considerou-se uma plataforma do tipo *Platform-as-a-Service* (e.g., Railway, Vercel ou Render), que oferece (i) *deploy* contínuo a partir do repositório Git, (ii) base de dados PostgreSQL gerida, e (iii) gestão segura das variáveis de ambiente (`DATABASE_URL`, `NEXTAUTH_SECRET`, `QR_SECRET`, `ALLOWED_EMAIL_DOMAIN`, credenciais de envio de email, etc.). O mesmo *Dockerfile* utilizado em desenvolvimento serve de base para a imagem de produção, garantindo paridade entre ambientes.

12 Conclusão

O Spot-On materializa uma solução completa para um problema concreto vivido diariamente na biblioteca da Universidade do Algarve. Do ponto de vista de engenharia de software, o projeto permitiu aplicar boas práticas em todas as fases: levantamento e priorização de requisitos sob SCRUM, desenho de uma arquitetura clara e justificada, implementação com tecnologias modernas e tipadas, persistência versionada com migrações, testes automatizados, *deployment* reprodutível e Integração Contínua.

Como trabalho futuro, identificam-se eixos como (i) a integração com sistemas de *notification push* para alertas em tempo real, (ii) a exposição de estatísticas agregadas de utilização para apoio à gestão da biblioteca e (iii) o alargamento do sistema a outros campi da Universidade.